



HE1025 Operativsystem

Ö3: Linux



Kursen...

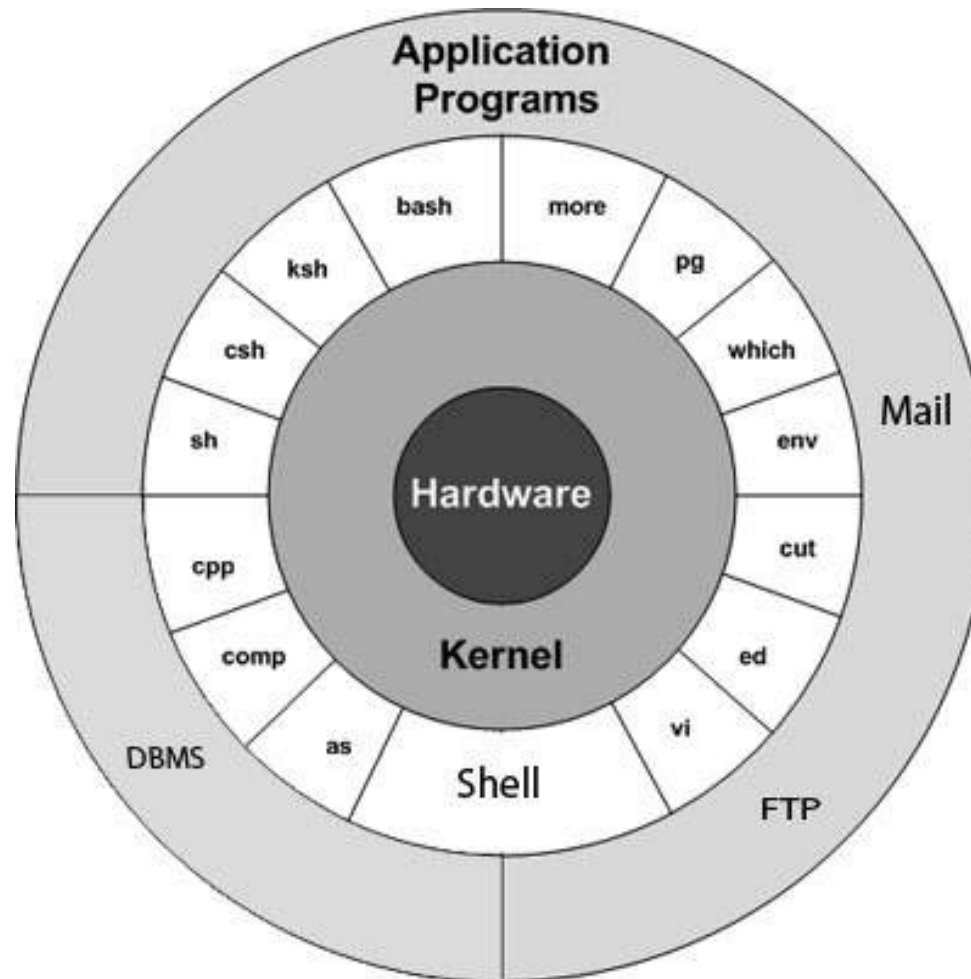
F1: V#1 Processorn.	Kap (1) 2..4 & 6
Ö1: V#2 c Pekare...	Kap 14 & c-boken
F2: V#3 Schemaläggning	Kap 7..9,(10-11), RTOS
L1: V#4 FreeRTOS	Lab #1 Handledning
F3: V#5 Segmentation	Kap 12..13, 15..17
Ö2: V#6 Buffer	t.b.d
F4: V#7 Paging	Kap 18-22, (23-24)
L2: V#8 Paging	Lab #2 Handledning
F5: C#1 Locks & CV	Kap (25) 26..27 & 28..29.1
Ö3: LINUX	Kap 5 (39)
F6: C#3 Semaphores	Kap 30..33, (34)
L3: C#4 Barberare	Lab #3 Handledning
F7: P#1 Storage	Kap (35,) 36, 40, 42 (37..39, 41 & 43..46)
Ö4: P#2 Buffer	t.b.d
F8: P#3 Communication	Kap (47, 39) & 48, (49..51)
L4: P#4 Client/Server	Lab #4 Handledning

TEN1: 4+4p/4:e-del, minst 4p per 4:e-del för E!

LAB1: Närvaro L1..4



Linux/Unix på 2*45 minuter...



Unix -69 AT&T
Thompson & Ritchie

Ett "enkelt" OS för
nya "minidatorer".

Kommandotolk CLI
ca 250 standardkom.

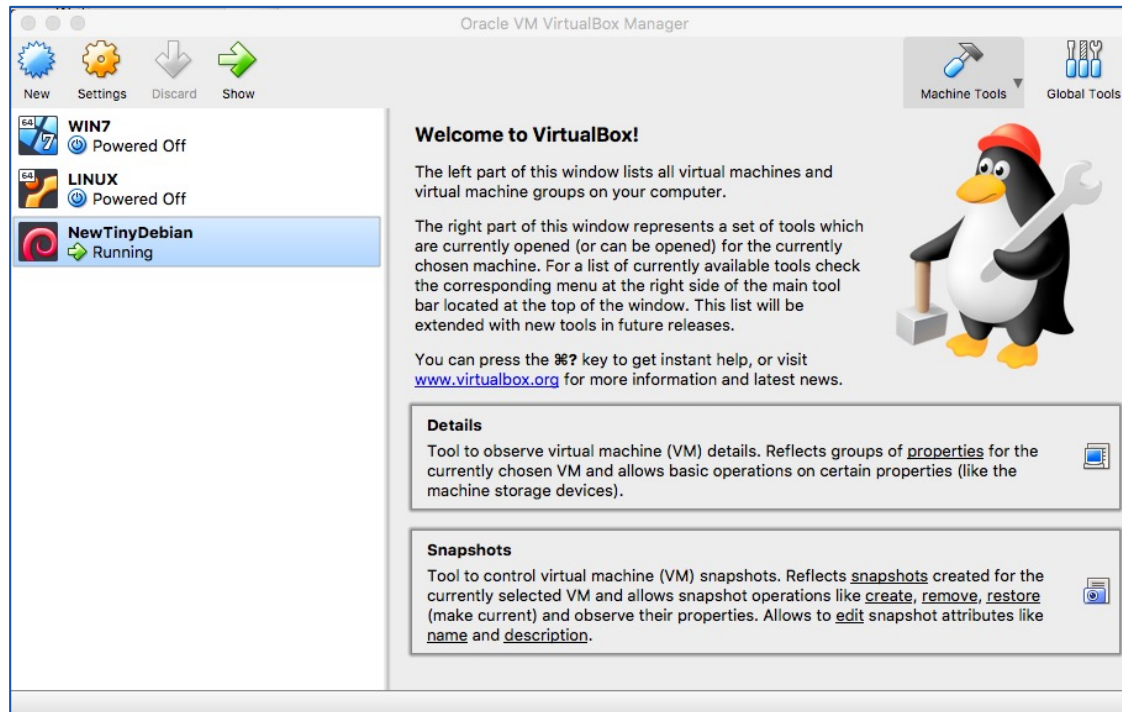
Linux -91 Hobbyprojekt
Linus Torvalds

Open Source!

Finns det en c-kompilator som kan generera kod för målmaskinen
så går det att skapa en Linux distribution för målmaskinen!



Installation...



!TESTA – TESTA – TESTA –TESTA – TESTA!

CASE

SENSITIVE!



Shell

whoami Who am I?
users
who
w

MNEMONICS –FLAGGOR ARGUMENT

```
me@newtinydebian:~$ whoami
me
me@newtinydebian:~$ users
me me
me@newtinydebian:~$ who
me      tty7      2018-12-01 12:52 (:0)
me      pts/0     2018-12-01 12:52 (:0)
me@newtinydebian:~$ w
13:58:59 up 1:16, 2 users, load average: 0,00, 0,00, 0,00
USER      TTY      FROM          LOGIN@      IDLE        JCPU       PCPU       WHAT
me        tty7      :0             12:52       1:16m      8:48       0.01s    /usr/bin/lxsess
me        pts/0     :0             12:52       0.00s      0.14s      0.00s    w
me@newtinydebian:~$
```




NAME

ls -- list directory contents

SYNOPSIS

ls [-ABCFGHLOPRSTUW@abcdefghiklmnopqrstuwx1] [file ...]

DESCRIPTION

For each operand that names a file of a information. For each operand that name tory, as well as any requested, associat

If no operands are given, the contents of operands are displayed first; directory

The following options are available:

- @ Display extended attribute keys
- l (The numeric digit ``one'') For nal.
- A List all entries except for . and
- a Include directory entries whose
- B Force printing of non-printable where xxx is the numeric value of
- b As -B, but use C escape codes wh
- C Force multi-column output; this

Betydelse

Rättigheter Eгна/Grp/Alla

-/R/W/X (eXecutable)

Antalet block i minnet

Vem som skapat filen

Vilken grupp den tillhör

Storlek i Byte

Skapad/Senast ändrad

Filens/Folderns namn

ted, associated within that direc-
given, non-directory al order.

is not to a termi-

le names as \xxx,

slash ('/') immediately after each pathname that is a directory, an asterisk ('*') after each that is exe-
n at sign('@') after each symbolic link, an equals sign('=') after each socket, a percent sign('%') after
out and a vertical bar '|' after each that is a FIFO.

(low.)

ne long (-l) format

options are speci-

abyte in order to

-k If the -s option is specified, print the file size allocation in kilobytes, not blocks. This option overrides the envi-

ronment variable BLOCKSIZE.

Typ	Beskrivning
-	Vanlig fil
b	Block fil (disk)
c	Char fil (disk)
d	Folder (directory)
l	"Alias"
p	Pipe IPC
s	Socket I/O

```
me@newtinydebian:~$ ls
bin  clproj  Desktop  GvR  GvRng_4.4
me@newtinydebian:~$ ls -l
totalt 20
drwxr-xr-x 2 me me 4096 11 apr 2012 bin
drwxr-xr-x 4 me me 4096 11 aug 20.00 clproj
drwxr-xr-x 2 me me 4096 15 apr 2012 Desktop
drwxr-xr-x 2 me me 4096 15 maj 2012 GvR
drwxr-xr-x 9 me me 4096 11 apr 2012 GvRng_4.4
```

-k If the -s option is specified, print the file size allocation in kilobytes, not blocks. This option overrides the environment variable BLOCKSIZE.



Metatecken och gömda filer...

Innehåller en folder många filer går det att söka med hjälp av mönstermatchning: * = ersätter 0..n tecken, ? = ersätter 1 tkn!

Den plats i folderhierarkin där du är förkortas '.'

Filer som börjar med '.' är normalt "osynliga"!

Folder ovanför (om den finns) förkortas '..'

Lägg till flaggan **-a**(ll) så visas dessa filer också!

```
me@newtinydebian:~$ ls -a
.          .config   .gvfs      .recently-used.xbel
..         .dbus     GvR        .thumbnails
.bash_history .ddd      GvRng_4.4  .w3m
.bash_logout Desktop    .gvnrgrc   .vboxclient-clipboard.pid
.bashrc      .dmrc     .kde       .vboxclient-display.pid
bin          .emacs.d  .local     .vboxclient-seamless.pid
clproj       .gconf    .mozilla   .Xauthority
.cache       .gconfd   .nano_history .xsession-errors
.codeblocks  .gnome2   .profile
```



pwd
.. / ~

För att flytta sig i folderhierarkin...

Kommandot för att byta folder är: **cd** **change directory**

cd .. Flytta upp till föräldrafoldern i hierarkin.

cd ~ Flytta dig till din hemfolder.

cd / Flytta dig till filsystemets root folder.

cd /dir/dir/.. En absolut angiven adress!

cd dir/dir/.. En relativt angiven adress (relativt där du är)!

pwd path working directory



mkdir
rmdir

Att hantera foldrar...

För att skapa en folder används kommandot:

mkdir **make directory**

mkdir lab	skapar folder lab i den folder du är i...
mkdir /lab	skapar folder lab i root-foldern!

För att ta bort en folder används kommandot:

rmdir **remove directory**

Foldern **MÅSTE** vara tom för att det ska fungera...

...se upp: wildcard är tillåtet i specifikationen...

...utpekade foldrar tas bort **UTAN ÅTERKOPPLING!**



Att skapa filer...

vi
emacs
cat
wc

Historiskt har texteditorerna "vi" och "emacs" använts...
...de använder en mycket speciell typ av semigrafik...
...för att ska en form av grafiskt gränssnitt...
...i dag används nog alltid olika utvecklingsmiljöer!

Använd VSCi kursen!

vi lab1 -> Hello World -> esc+z esc+z -> Filen skapad!
emacs lab2 -> Hello Again -> ctr+x ctr+s -> ctr+x ctr+c -> ok!

cat lab1 -> Hello World

concatenate and print file

wc lab1 -> 1 2 12 lab1 [rader ord byte namn]

(word count)



vi

<https://www.tutorialspoint.com/unix/unix-vi-editor.htm>



emacs

<https://www.gnu.org/software/emacs/tour/>



Att hantera filer...

cp
mv
rm

För att kopiera en fil:

cp source duplicate

copy

För att byta namn på en fil:

mv old new

move

För att ta bort en fil (inte hela sanningen):

rm name

remove



Filer & Rättigheter...

Via t.ex. **ls -l** går det att se en fils rättigheter...

...på ägare/user/u, grupp/group/g & andra/other(o...

...med valen **-(0)/read(4)/write(2)/execute(1)**

Rättigheterna kan ändras med kommandot:

chmod **change mode** på två sätt:

Symboliskt +(add)/-(remove)/=(set): **chmod u=rwx, g=rx, o=r**

Absolut bitmönster: **chmod 754**

sudo command

password



Input and Output...

Unix/Linux har normalt 3 filer "öppna"...

stdin (standard input) med filid 0, normalt till för indata.

stdout (standard output) med filid 1, normalt till för utdata.

stderr (standard error) med filid 2, normalt till för felmeddel.

Alla kommandon vi stött på läser från stdin...

...och skriver till stdout...

...men det går lätt att ändra på med < << | >> >



Input and Output...

who > users

Utskriften hamnar i (nya) filen users!

who >> users

Utskriften läggs till i slutet på filen users!

wc < users

wc tar sin indata från filen users!

wc << EOF

wc will read from stdin until it finds an EOF

who | **wc**

styr utdatat från who som indata till wc!



Vad finns i en fil..

`head [-<lines>] file` Visar 10 (lines) första raderna...

`tail [-<lines>] file` Visar 10 (lines) sista raderna...

`more` eller `less` Visa innehållet i en fil...

`grep 'reg.exp' filename` Visa alla rader med reg.exp!

Otroligt vanligt tillsammans med
`|` och `>`

(echo "text" Skriver ut argumenten till standard output)



File System API

```
int fd = open("foo", O_CREAT|O_WRONLY|O_TRUNC, S_IRUSR|S_IWUSR);
```

```
int rc = write(fd, buffer, size);
```

```
int rc = read(fd, buffer, size);
```

```
rc = fsync(fd);
```

```
stat()
```

(Shell stat <file>)

```
struct stat {
```

```
    dev_t    st_dev;    /* ID of device containing file */
```

```
    ino_t    st_ino;    /* inode number */
```

```
    mode_t   st_mode;   /* protection */
```

```
    → nlink_t st_nlink; /* number of hard links */
```

```
    uid_t    st_uid;    /* user ID of owner */
```

```
    gid_t    st_gid;    /* group ID of owner */
```

```
    dev_t    st_rdev;   /* device ID (if special file) */
```

```
    off_t    st_size;   /* total size, in bytes */
```

```
    blksize_t st_blksize; /* blocksize for filesystem I/O */
```

```
    blkcnt_t st_blocks; /* number of blocks allocated */
```

```
    time_t   st_atime;  /* time of last access */
```

```
    time_t   st_mtime;  /* time of last modification */
```

```
    time_t   st_ctime;  /* time of last status change */
```

```
};
```

Hard link: In filename alias

Symbolic: In -s filename alias

ref count +1, not to dir or other part.

skapar filen alias med path till filen!



Processer...

Allt som sker i LINUX/UNIX utförs av **processer**...

...för att utföra kommandot **ls** startas en process...

...alla processer har ett unikt 5-siffrigt processor id (pid)...

...som kan tas fram via kommandot **ps -f process status!**

```
Oscars-MBP:~ OC$ ps -f
  UID    PID  PPID  C  STIME  TTY      TIME CMD
  501   8473   8472  0  6:18pm ttys000    0:00.03 -bash
Oscars-MBP:~ OC$
```

...kommandon kan utföras i förgrunden (foreground) där CLI blir blockerad tills kommandot slutförts (Control+C = quit)...

...men det går också att starta processer i bakgrunden...

1	UID User ID that this process belongs to (the person running it)
2	PID Process ID
3	PPID Parent process ID (the ID of the process that started it)
4	C CPU utilization of process
5	STIME Process start time
6	TTY Terminal type associated with the process
7	TIME CPU time taken by the process
8	CMD The command that started this process



Processer...

För att starta en process i bakgrunden...

- ...lägg till ett '&' efter kommandot...

- ...CLI blir inte blockerat, så fler kommandon kan utföras!

För att avsluta en bakgrundsprocess...

- ...använd ps kommandot för att hitta process id...

- ...**kill** pid (om det inte hjälper **kill -9** pid)

Alla processer i UNIX/LINUX har en "förälder"...

- ...utom den första processen som heter **init**...

- ...skulle föräldern till en process dö...

- ...blir processen föräldralös (orphan) och får init som f!

Processer som hör till kommandon som startas via CLI...

...har vanligtvis shellet som sin förälder.



Processer...

Orphan kill -9 pid

Alla processer returnerar alltid någon form av status...

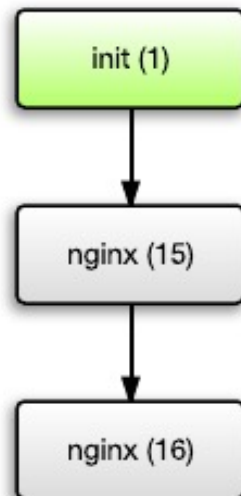
...skulle en process avslutas,

innan föräldern tagit del av resultatet...

...så är processen död, men finns, en s.k. **Zombie!**

...tas bort genom att ta bort föräldern, om inte init = reboot!

Stage 1: Nginx (PID 15)
creates child process



Stage 2: Nginx (PID 15) exits.
Its child process (PID 16) no
longer has a parent and is
now "orphaned"



Stage 3: Since PID 16 no longer
has a parent, it is "adopted" by
the init process, which now
becomes its parent





Processer...

Ibland vill vi starta en process som ska gå för “evigt” och bistå med någon speciell uppgift...

...en sådan process kallas för en daemon och skapas genom att tillverka en orphan-proces som därefter kopplas lös från ansluten terminal!

ps -axj (all, no tty, job)



UID	PID	PPID	PGID	SID	TTY	CMD
root	1	0	1	1	?	/sbin/init
root	2	0	0	0	?	[kthreadd]
root	3	2	0	0	?	[ksoftirqd/0]
root	6	2	0	0	?	[migration/0]
root	7	2	0	0	?	[watchdog/0]
root	21	2	0	0	?	[cpuset]
root	22	2	0	0	?	[khelper]
root	26	2	0	0	?	[sync_supers]
root	27	2	0	0	?	[bdi-default]
root	29	2	0	0	?	[kblockd]
root	35	2	0	0	?	[kswapd0]
root	49	2	0	0	?	[scsi_eh_0]
root	256	2	0	0	?	[jbd2/sda5-8]
root	257	2	0	0	?	[ext4-dio-unwrit]
syslog	847	1	843	843	?	rsyslogd -c5
root	906	1	906	906	?	/usr/sbin/cupsd -F
root	1037	1	1037	1037	?	/usr/sbin/inetd
root	1067	1	1067	1067	?	cron
Daemon	1068	1	1068	1068	?	atd
root	8196	1	8196	8196	?	/usr/sbin/sshd -D
root	13047	2	0	0	?	[kworker/1:0]
root	14596	2	0	0	?	[flush-8:0]
root	26464	1	26464	26464	?	rpcbind -w
statd	28490	1	28490	28490	?	rpc.statd -L
root	28553	2	0	0	?	[rpciod]
root	28554	2	0	0	?	[nfsiod]
root	28561	1	28561	28561	?	rpc.idmapd
root	28761	2	0	0	?	[lockd]
root	28764	2	0	0	?	[nfsd]



Processen döps ofta till något som slutar på d...

...nfsd = Network File System Daemon!



Process API

UNIX/Linux erbjuder c-programmeraren via unistd.h api:et tillgång till underliggande POSIX OS API...

...via API:et går det bl.a. att skapa och hantera processer!

Funktionen: **pid_t fork(void);** skapar en ny process.

Det är verkligen en ny process men addressrymd, register, filpekare etc är en KOPIA av förälderns. M.a.o kommer bägge att fortsätta där den var när anropet returnerar. Är det returnerade värdet <0 så blev det fel, är det >0 så är det PID för barnet, är det 0 så är det barnprocessen som kör...

...barnet "vet inte" att det är ett barn på annat sätt än att den returnerade koden är 0!



fork()

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  int
6  main(int argc, char *argv[])
7  {
8      printf("hello world (pid:%d)\n", (int) getpid());
9      int rc = fork();
10     if (rc < 0) {                // fork failed; exit
11         fprintf(stderr, "fork failed\n");
12         exit(1);
13     } else if (rc == 0) {        // child (new process)
14         printf("hello, I am child (pid:%d)\n", (int) getpid());
15     } else {                    // parent goes down this path (main)
16         printf("hello, I am parent of %d (pid:%d)\n",
17               rc, (int) getpid());
18     }
19     return 0;
20 }
```

API

```
int      dup2(int, int);
void     _exit(int);
void     encrypt(char [64], int);
int      execl(const char *, const char *, ...);
int      execle(const char *, const char *, ...);
int      execlp(const char *, const char *, ...);
int      execlp(const char *, const char *, ...);
int      execve(const char *, char *const [], char *const []);
int      execvp(const char *, char *const []);
int      faccessat(int, const char *, int, int);
int      fchdir(int);
int      fchown(int, uid_t, gid_t);
int      fchownat(int, const char *, uid_t, gid_t, int);
int      fdatsync(int);
int      fexecve(int, char *const [], char *const []);
pid_t    fork(void);
long     fpathconf(int, int);
```

```
prompt> ./p1
hello world (pid:29146)
hello, I am parent of 29147 (pid:29146)
hello, I am child (pid:29147)
prompt>
```

Alltid på detta sätt?!



fork()

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  int main(void){
6      printf("Parent starts (pid:%d)\n",(int)getpid());
7      if (fork()) {
8          printf("Parent continues (pid:%d)\n",(int)getpid());
9      } else {
10         printf("Child starts (pid:%d)\n",(int)getpid());
11     }
12     printf("Parent or Child done (pid:%d)\n",(int)getpid());
13 }
```

```
n166-p66:vsws ac$ gcc fork.c
n166-p66:vsws ac$ ./a.out
Parent starts (pid:4782)
Parent continues (pid:4782)
Parent or Child done (pid:4782)
Child starts (pid:4783)
Parent or Child done (pid:4783)
n166-p66:vsws ac$
```



wait(), waitpid(#)



```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  int
7  main(int argc, char *argv[])
8  {
9      printf("hello world (pid:%d)\n", (int) getpid());
10     int rc = fork();
11     if (rc < 0) {          // fork failed; exit
12         fprintf(stderr, "fork failed\n");
13         exit(1);
14     } else if (rc == 0) { // child (new process)
15         printf("hello, I am child (pid:%d)\n", (int) getpid());
16     } else {              // parent goes down this path (main)
17         int wc = wait(NULL);
18         printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
19               rc, wc, (int) getpid());
20     }
21     return 0;
22 }
```



```
prompt> ./p1
hello world (pid:29146)
hello, I am child (pid:29147)
hello, I am parent of 29147 (pid:29146)
prompt>
```

Alltid denna ordning!



wait(), waitpid(#)

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  int main(void){
6      printf("Parent starts (pid:%d)\n", (int)getpid());
7      if (fork()) {
8          printf("Parent continues (pid:%d)\n", (int)getpid());
9          wait(NULL); ←
10     } else {
11         printf("Child starts (pid:%d)\n", (int)getpid());
12     }
13     printf("Parent or Child done (pid:%d)\n", (int)getpid());
14 }
```

```
n166-p66:vsws ac$ gcc fork.c
n166-p66:vsws ac$ ./a.out
Parent starts (pid:4843)
Parent continues (pid:4843)
Child starts (pid:4844)
Parent or Child done (pid:4844)
Parent or Child done (pid:4843)
n166-p66:vsws ac$
```




execvp()

```
prompt> ./p3
hello world (pid:29383)
hello, I am child (pid:29384)
      29      107      1030 p3.c
hello, I am parent of 29384 (wc:29384) (pid:29383)
prompt>
```

[strdup() är en variant av strcpy()]



```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <sys/wait.h>
6
7  int
8  main(int argc, char *argv[])
9  {
10     printf("hello world (pid:%d)\n", (int) getpid());
11     int rc = fork();
12     if (rc < 0) {          // fork failed; exit
13         fprintf(stderr, "fork failed\n");
14         exit(1);
15     } else if (rc == 0) { // child (new process)
16         printf("hello, I am child (pid:%d)\n", (int) getpid());
17         char *myargs[3];
18         myargs[0] = strdup("wc");    // program: "wc" (word count)
19         myargs[1] = strdup("p3.c"); // argument: file to count
20         myargs[2] = NULL;           // marks end of array
21         execvp(myargs[0], myargs);  // runs word count
22         printf("this shouldn't print out");
23     } else {                  // parent goes down this path (main)
24         int wc = wait(NULL);
25         printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
26               rc, wc, (int) getpid());
27     }
28     return 0;
29 }
```



Bakgrund till att fork() & exec() är delade...

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <fcntl.h>
6  #include <sys/wait.h>
7
8  int
9  main(int argc, char *argv[])
10 {
11     int rc = fork();
12     if (rc < 0) {          // fork failed; exit
13         fprintf(stderr, "fork failed\n");
14         exit(1);
15     } else if (rc == 0) { // child: redirect standard output to a file
16         close(STDOUT_FILENO);
17         open("./p4.output", O_CREAT|O_WRONLY|O_TRUNC, S_IRWXU);
18
19         // now exec "wc"...
20         char *myargs[3];
21         myargs[0] = strdup("wc"); // program: "wc" (word count)
22         myargs[1] = strdup("p4.c"); // argument: file to count
23         myargs[2] = NULL;          // marks end of array
24         execvp(myargs[0], myargs); // runs word count
25     } else {                  // parent goes down this path (main)
26         int wc = wait(NULL);
27     }
28     return 0;
29 }
```

```
prompt> ./p4
prompt> cat p4.output
      32      109      846 p4.c
prompt>
```



Lab 3!

Zoom-session med närvarokontroll 8.15..12.00

Ni löser uppgifter efterhand i grupper...

...alla som deltar kommer att bli godkända.

De som har ett RISC-V system ska ha det i beredskap!



Mindmap

whoami

users, who, w

man, ls (-a -l)

*** ? . .. / ~ cd**

pwd, mkdir, rmdir

vi, emacs

cat, wc

cp, mv, rm

chmod

< << | >> >

head, tail

more, less

grep

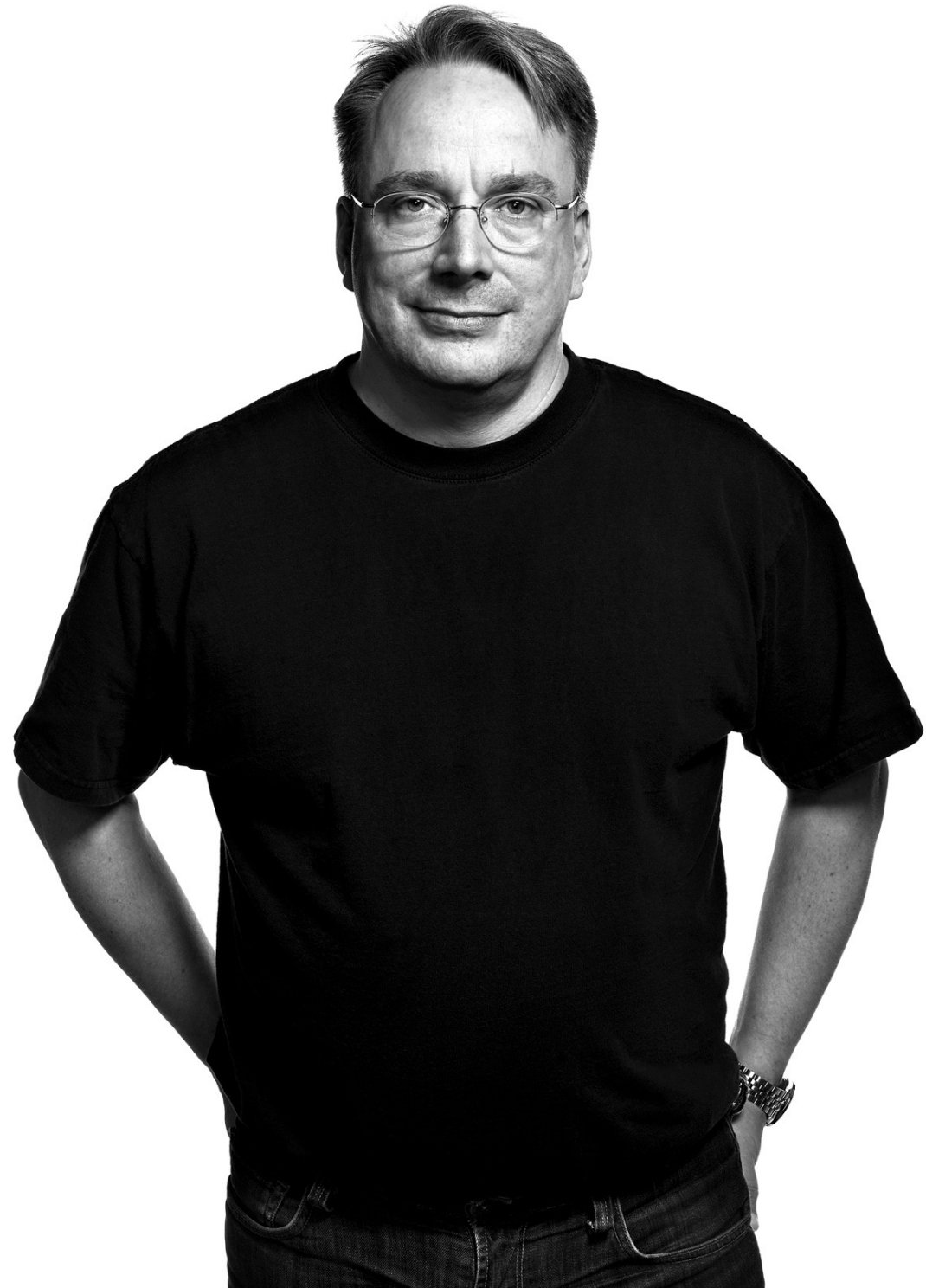
ln, ps, kill

Pid, init

Orphant, Zombie

fork, wait

execvp





The Sleeping Barber Problem...

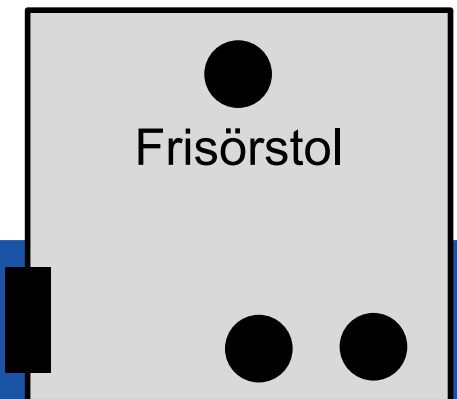
Tänk dig en frisörsalong...

...det finns en frisör och frisörens arbetsstol,
frisören behöver i snitt 15 min för att klippa en kund...

...rummet har också 2 stolar där kunder kan vänta,
kunder anländer i snitt var 20 min,
väntar redan 2 kunder går kunden direkt...

...finns ingen kund, sover frisören i sin stol!

Dörr





Poisson process, M/M/G köer...

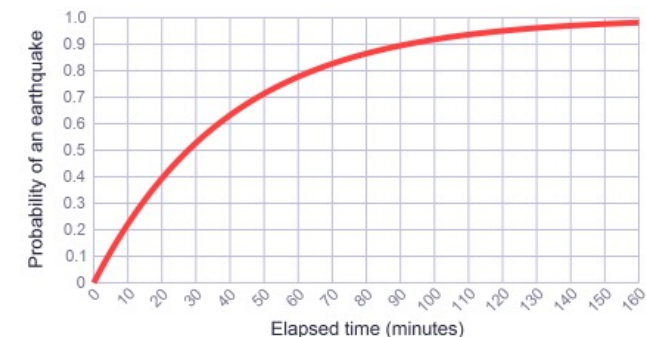
Att “klienter” med någon regelbundenhet efterfrågar tjänster från “konsumenter” är väldigt vanligt i moderna program...

Klienter efterfrågar tjänster med någon snittintensitet...
...men tiden mellan olika förfrågningar är helt oberoende...
...tiden är med andra ord inte normalfördelad...
...utan fördelad enligt en så kallad Poisson fördelning...
...all denna insikt utgör basen för så kallad kö-teori...
...som är en viktig del av modern programmeringsinsikt!

$\lambda = \frac{1}{40}$ and call it the *rate parameter*.

...och berättar i det här fallet att vi förväntar oss att något kommer att hända i snitt var fyrtionde minut!

$$F(x) = 1 - e^{-\lambda x}$$





Template code running...

The screenshot shows a code editor window titled "main.c [barber] - Code::Blocks 10.05" and a terminal window titled "barber".

Code Editor (main.c):

```
5
6 #define WTIME 60*8 // Shop open 8h
7 #define ETIME 60*7 // No new customers last hour
8 #define SCALE 8 // 1 minute is 1/SCALE second
9
10 int delta(float lambda) {
11 // Knuth imp of poisson process with extrem value truncation
12 return (int)(-log(((rand())/1.5f)/RAND_MAX)+0.20f)/lambda;
13 };
14
15 int main(void){
16 int at=0, wait, id=0;
17 printf("Barber shop simulation 0.1 AC\n");
18
19 // Init needed concurrency features...
20
21 // Create the barber!
22 srand((unsigned)time(NULL));
23
24
25 do {
26 wait=delta(1/20.0f); at+=wait;
27 sleep(wait/SCALE);
28 printf("%02d:%02d !!! New customer #%02d enters shop... \n", at/60
29 // Create new customer
30 } while (at<ETIME); // No more customers (roughly) last hour!
31
32 sleep((WTIME-at)/SCALE); // Barber works on remaining customers...
33
34 printf("Barber shop simulation done!\n");
35 return 0;
36 };
37
```

Terminal (barber):

```
00:08 !!! New customer #00 enters shop...
00:24 !!! New customer #01 enters shop...
00:27 !!! New customer #02 enters shop...
00:42 !!! New customer #03 enters shop...
01:03 !!! New customer #04 enters shop...
01:30 !!! New customer #05 enters shop...
01:36 !!! New customer #06 enters shop...
01:42 !!! New customer #07 enters shop...
02:00 !!! New customer #08 enters shop...
02:11 !!! New customer #09 enters shop...
02:22 !!! New customer #10 enters shop...
02:46 !!! New customer #11 enters shop...
02:54 !!! New customer #12 enters shop...
02:59 !!! New customer #13 enters shop...
03:09 !!! New customer #14 enters shop...
03:17 !!! New customer #15 enters shop...
03:40 !!! New customer #16 enters shop...
03:47 !!! New customer #17 enters shop...
04:08 !!! New customer #18 enters shop...
04:12 !!! New customer #19 enters shop...
04:40 !!! New customer #20 enters shop...
04:44 !!! New customer #21 enters shop...
05:12 !!! New customer #22 enters shop...
05:20 !!! New customer #23 enters shop...
05:26 !!! New customer #24 enters shop...
05:43 !!! New customer #25 enters shop...
05:56 !!! New customer #26 enters shop...
06:02 !!! New customer #27 enters shop...
06:09 !!! New customer #28 enters shop...
06:14 !!! New customer #29 enters shop...
06:35 !!! New customer #30 enters shop...
06:50 !!! New customer #31 enters shop...
07:13 !!! New customer #32 enters shop...
Barber shop simulation done!

Process returned 0 (0x0) execution time : 43.043 s
Press ENTER to continue.
```

Logs & others:

```
Checking for existence: /home/me/barber/bin/Debug/barber
Executing: xterm -T barber -e /usr/bin/cb_console_runner LD_LIBRARY_PATH=
$LD_LIBRARY_PATH: /home/me/barber/bin/Debug/barber (in /home/me/barber/.)
```



Historik

181201 0.5 AC Draft

190127 1.0 AC Release 1.0

210428 1.1 AC Anpassad RISC-V